

Assessment Module

Solid Design Specification

Instructor Quiz Creation + Assignment Evaluation

Architecture: React → API → Express Route → Middleware → Controller → Model → MySQL

Purpose

Define the professional design before implementation. This document covers responsibilities, database mapping, API contracts, security, UI/UX, validation, testing, and implementation order.

Team rule: use the existing Model-based backend structure. No Repository layer is introduced.

Features

- Instructor Quiz Builder: create, edit, save draft, validate, and publish quizzes.
- Assignment Evaluation: list submissions, inspect files, grade, and provide feedback.

1. Architecture & Team Alignment

The agreed backend flow is:

Route → Middleware → Controller → Model → MySQL

The frontend flow is:

React Page → API Service → REST API → Backend

Team	Responsibility
System Analysis	Requirements, business rules, workflows, permissions, acceptance criteria.
System Design / UI/UX	Responsive page structure, interaction, states, validation feedback.
Backend Developer	Routes, middleware, controllers, models, validation, transactions, security.
Frontend Developer	React pages/components, API integration, state, loading/error/success UI.

Module boundary

Assessment is one domain with two independent feature flows: Quiz Management and Assignment Evaluation. They share authentication and authorization patterns but do not share unnecessary database structures.

Assessment ■■■ Quiz Management ■ ■■■ Builder ■ ■■■ Questions / Options ■ ■■■ Draft / Published lifecycle ■ ■■■ Attempts / Results ■■■ Assignment Evaluation ■■■ Submission List ■■■ Submission Review ■■■ Files ■■■ Grade / Feedback

2. Database Design

Use the existing normalized schema. Do not create duplicate quiz, grading, or submission tables.

Quiz relationships

courses 1 ■■■ N quizzes quizzes 1 ■■■ N questions questions 1 ■■■ N question_options quizzes 1 ■■■ N quiz_attempts quiz_attempts 1 ■■■ N quiz_answers

Table	Role / important fields
quizzes	course_id, lesson_id, title, description, total_questions, time_limit_minutes, pass_percent, status
questions	quiz_id, question_number, prompt, code, difficulty, points
question_options	question_id, option_key, option_text, is_correct
quiz_attempts	quiz_id, user_id, started_at, submitted_at, score, percentage, status
quiz_answers	attempt_id, question_id, selected_option_id, is_correct, points_earned

QuizBuilder compatibility enhancement

If the current database does not already contain these fields, add only the following:

```
ALTER TABLE quizzes ADD COLUMN shuffle_questions BOOLEAN NOT NULL DEFAULT FALSE; ALTER TABLE questions ADD COLUMN question_type VARCHAR(30) NOT NULL DEFAULT 'multiple_choice', ADD COLUMN explanation TEXT NULL;
```

Assignment relationships

assignments 1 ■■■ N submissions submissions 1 ■■■ N submission_files

Table	Role / important fields
assignments	course_id, lesson_id, title, instructions, due_date, points, status
submissions	assignment_id, user_id, comment, submitted_at, score, instructor_comment, graded_at, status
submission_files	submission_id, original_name, stored_name, file_path, file_size, mime_type

3. Quiz Builder — UX & Business Rules

Instructor workflow

Open Builder → Select Course/Lesson → Enter Settings → Add Questions → Add Options → Select Correct Answer → Validate → Save Draft or Publish

Recommended page structure

Top bar: Back | Quiz Builder | Student Preview | Save Draft | Publish

Settings: Title | Course | Lesson/Module | Time Limit | Passing Score | Shuffle Questions

Question card: Number | Type | Points | Prompt | Options | Correct Answer | Explanation | Delete

Actions: Add Question | Add Option | Duplicate Question | Delete Question

Rule	Requirement
Ownership	Instructor may manage quizzes only for courses they own; admin has broader access.
Title / Course	Required.
Questions	At least one before publishing.
Options	Multiple-choice must have valid options and one correct answer for v1.
Points	Positive numeric value.
Time limit	Positive integer or explicit unlimited policy.
Passing score	Valid percentage, 0–100.
Publish	Incomplete quizzes remain drafts.

Lifecycle

DRAFT → PUBLISHED → ARCHIVED

A published quiz should not be destructively changed if students already have attempts. If versioning becomes necessary, add it as a later architectural decision rather than over-engineering v1.

4. Assignment Evaluation — UX & Business Rules

Submission list

Filters: Course | Assignment | Status | Search Student Columns: Student | Assignment | Submitted | Status | Grade | Action

Submission review

Student → Assignment → Submitted date → Files → Student comment → Grade → Feedback → Save Grade

Rule	Requirement
Access	Authenticated instructor/admin only.
Ownership	Instructor must own the course/assignment associated with the submission.
Score	$0 \leq \text{score} \leq \text{assignment.points}$.
Feedback	Validated text; required/optional according to final product rule.
Status	Successful grading sets status = graded and records graded_at.
Files	Only authorized submission files may be accessed.
Regrading	Allow controlled updates; history can be added later if required.

State model

SUBMITTED → GRADED → UPDATED / RE-GRADED

Do not create a separate assignment_grades table for v1 because the existing submission record already owns the current score, feedback, grading timestamp, and status.

5. API Contract

Quiz endpoints

Method	Endpoint	Purpose
POST	/api/quizzes	Create quiz with questions/options.
GET	/api/quizzes/:id	Retrieve quiz according to authorization.
PATCH	/api/quizzes/:id	Update editable quiz data.
DELETE	/api/quizzes/:id	Delete/archive according to lifecycle.
PATCH	/api/quizzes/:id/publish	Publish validated draft.

Create quiz request

```
{ "courseId": 12, "lessonId": 45, "title": "Module 4 Assessment", "timeLimitMinutes": 45,
  "passPercent": 75, "shuffleQuestions": true, "questions": [ { "prompt": "What is the main purpose
of CAP theorem?", "questionType": "multiple_choice", "points": 10, "explanation": "Explains the
trade-off.", "options": [ {"optionKey":"A","optionText":"Choice A","isCorrect":false},
{"optionKey":"B","optionText":"Choice B","isCorrect":true} ] } ] }
```

Assignment endpoints

Method	Endpoint	Purpose
GET	/api/assignments/submissions	List authorized instructor submissions.
GET	/api/assignments/submissions/:id	Retrieve one submission and files.
PUT	/api/assignments/submissions/:id/grade	Assign/update grade and feedback.

Grade request

```
{ "score": 85, "feedback": "Good implementation. Improve validation and error handling." }
```

6. Backend File Design

Target structure, while reusing existing files where they already exist:

```
backend/src/ ■■■ controllers/ ■ ■■■ quizController.js ■ ■■■ assignmentController.js ■■■ models/
■ ■■■ Quiz.js ■ ■■■ Assignment.js ■■■ routes/ ■ ■■■ quizRoutes.js ■ ■■■ assignmentRoutes.js
■■■ middleware/ ■ ■■■ authMiddleware.js ■■■ validators/ ■■■ quizValidator.js ■■■
assignmentValidator.js
```

No Repository layer. Models own parameterized SQL and database operations, consistent with the team's structure.

Route

Defines HTTP endpoints and connects middleware/controllers. It should contain no SQL.

Middleware

Verifies JWT, establishes req.user, checks role, and performs request validation where appropriate.

Controller

Coordinates request data, invokes the model, and returns standardized responses. It should not contain large SQL blocks.

Model

Owens database queries, joins, transactions, and data persistence. Quiz creation must use a transaction because multiple related tables are written together.

Frontend structure

```
src/ ■■■ pages/instructor/ ■ ■■■ QuizBuilder.jsx ■ ■■■ AssignmentGrading.jsx ■■■
components/instructor/ ■ ■■■ QuizSettings.jsx ■ ■■■ QuizQuestionCard.jsx ■ ■■■
SubmissionTable.jsx ■ ■■■ GradeSubmissionForm.jsx ■■■ services/ ■■■ quizApi.js ■■■
assignmentApi.js
```

7. Security & Validation

Backend authorization is mandatory. Hiding a frontend button is not security.

Authorization flow

```
JWT → authMiddleware → role check → resource ownership check → controller → model
```

- Instructor can modify only assessments belonging to courses they own.
- Admin may have broader management privileges.
- Submission file access must also verify ownership/authorization.
- Use parameterized SQL; never interpolate user input into SQL.
- Use express-validator according to the project's validation convention.

Validation matrix

Input	Validation
IDs	Valid positive/expected identifier format.
Quiz title	Required, trimmed, reasonable maximum length.
Question	Required prompt; valid question type.
Options	Non-empty option text; correct-answer rules enforced.
Points	Positive number.
Time limit	Positive integer if provided.
Passing score	0–100.
Grade	0 through assignment maximum points.
Feedback	Validated length/content; safe response encoding.

Error response

```
{ "success": false, "message": "Validation failed", "errors": [] }
```

Use the existing global error middleware for 400/401/403/404/409/422/500 handling. Do not expose SQL errors, secrets, stack traces, or internal paths in production responses.

8. Data Flow & Transactions

Quiz creation

```
QuizBuilder.jsx → quizApi.js → POST /api/quizzes → quizRoutes → auth/validation → quizController  
→ Quiz model → MySQL
```

The model performs one transaction:

```
BEGIN → INSERT quiz → INSERT questions → INSERT options → update total_questions → COMMIT  
Failure → ROLLBACK
```

Assignment grading

```
AssignmentGrading.jsx → assignmentApi.js → PUT /api/assignments/submissions/:id/grade → route →  
middleware → controller → Assignment model → MySQL
```

Grade operation

Validate score → find submission → verify assignment ownership → update score/feedback/graded_at/status → return standardized response.

Consistency principle

The database is the source of truth. React state reflects API responses; it must not assume a save succeeded until the backend confirms it.

9. Testing & Definition of Done

Quiz testing

1. Instructor login → obtain JWT. 2. Create/identify owned course. 3. Create valid quiz. 4. Verify quizzes/questions/options in MySQL. 5. Retrieve quiz. 6. Update draft. 7. Publish. 8. Test invalid title, points, options, correct answer, unauthorized course. 9. Verify status codes and response shape.

Assignment grading testing

1. Instructor login. 2. Identify owned assignment. 3. Ensure student submission exists. 4. List submissions. 5. Open submission and files. 6. Grade valid submission. 7. Verify score, instructor_comment, graded_at, status in MySQL. 8. Test score above maximum. 9. Test another instructor's submission. 10. Verify authorization/validation failures.

Frontend acceptance criteria

- Quiz draft saves all metadata, questions, and options.
- Publish rejects incomplete quizzes.
- Preview is read-only.
- Instructor sees only authorized submissions.
- Grade and feedback persist after refresh.
- Loading, empty, validation, API-error, success, and saving states are handled.
- Responsive on mobile and desktop.

Definition of Done

A feature is complete only when UI, API, authorization, validation, persistence, error handling, and tests work together. A mock-only page is not considered complete.

10. Implementation Roadmap & Decisions

Step	Implementation
1	Inspect existing models/routes and confirm current database columns.
2	Apply only missing quiz schema columns, if needed.
3	Implement Quiz model methods and transaction boundary.
4	Implement quiz controller, routes, middleware/validation integration.
5	Connect QuizBuilder to the real API.
6	Test quiz creation/update/publish in Postman and MySQL.
7	Implement Assignment model operations for submissions/grading.
8	Implement assignment controller/routes/validation.
9	Build and connect AssignmentGrading.jsx.
10	Run end-to-end security, validation, responsive UI, and regression tests.

Architectural decisions

Decision	Reason
Existing MySQL schema	Avoid duplicate assessment tables and preserve current project.
Model-based backend	Matches the team's established architecture; no Repository layer.
Separate quiz and assignment flows	Clear responsibilities and easier maintenance.
Quiz transaction	Prevents partial quiz/question/option creation.
Backend ownership checks	Prevents cross-instructor access.
No grade-history table in v1	Avoid over-engineering until audit history is a real requirement.

Final architecture

Instructor UI → React components → API service → Express route →
Authentication/Authorization/Validation → Controller → Model → MySQL → Standard API response →
React state

This design is the baseline. Code should be written against this contract rather than changing architecture file-by-file during implementation.